Name: Riya Ravikkumar

Student ID: 226415795

Unit Name: Machine Learning

Unit Code: SIT720

Assignment: 11.1 HD TASK (Machine Learning Research)

Reproduction and Improvement of an efficient stacking-based ensemble technique for early heart attack prediction

1. Introduction

In this document, a replication and an extension of a machine learning research article on early heart attack prediction based on stacking-based ensemble classification is conducted [1]. In part 1, the experimental methodology from the original research article is replicated as closely as possible according to the provided description in the article. In part 2, a real limitation of the methodology used in the original article was found and addressed.

2. Reproduction of the Original Paper

2.1 Research Problem and Motivation

Heart diseases have been cited as the top cause of deaths worldwide, and accurate risk prediction of this disease could be highly beneficial for preventive measures [1]. In this paper, an investigation has been conducted to see if ensemble of multiple classifiers using stacking methodology could increase the accuracy of prediction of heart attacks.

2.2 State-of-the-Art and Research Gap

The literature study focuses on past works in predicting heart diseases using individual algorithms (Random Forest, SVM, KNN, Decision Tree, Naïve Bayes, Logistic Regression) as well as simple ensemble algorithms like majority vote, and the accuracies reported in past works for the Cleveland data sets are found in the range of 85-93%. The gap found in this study is that most past research works concentrate on individual algorithm analysis but not the meta-classifier learning approach.

2.3 Dataset and Features

The paper utilizes 'Public Health Dataset' from Kaggle which is mentioned to be a merge of data from four different sources (Cleveland, Hungary, Switzerland, and Long Beach V) and has

1025 records with 14 clinical features (age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise induced angina, ST depression (oldpeak), slope, number of major vessels, thalassemia and the binary target)."

We used the exact same Kaggle dataset (johnsmith88/heart-disease-dataset). Here is the output for dataset information, feature preview, class distribution, and missing value check:

```
(1025, 14)
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0
target
1    526
0    499
Name: count, dtype: int64
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
...
ca          0
thal        0
target      0
dtype: int64
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

2.4 Methodology and Experimental Protocol

The paper uses: (i) missing value imputation (unnecessary in this case, since there are no missing values), (ii) feature scaling, (iii) splitting of the dataset into training and test samples in the proportion of 80%/20%, and (iv) six algorithms – Logistic Regression, Decision Tree, Random Forest, XGBoost, Naive Bayes, K-Nearest Neighbors, as well as stacking ensemble with all six combined using a meta-classifier.

Assumption: The paper does not mention the ratio in which the train and test sets have been split. An 80/20 split with the fixed random state (random_state=42) and stratification by target variable was assumed, as per common practice in the reviewed papers. It should be noted that this information is provided as an assumption, not as a paper statement.

In the context of the meta-classifier for the stacking algorithm, Logistic Regression was chosen as the final classifier with the internal 5-fold cross-validation, as described in scikit-learn's StackingClassifier.

Below is the output confirming the train/test split sizes and class balance in each:

```
(820, 13) (205, 13)
target
1    421
0    399
Name: count, dtype: int64
target
1    105
0    100
Name: count, dtype: int64
```

2.5 Evaluation Metrics

Consistent with both the task requirements and the original paper, we report Accuracy, Precision, Recall, F1-score, and AUC for every classifier.

2.6 Published Results (from the Paper)

The paper reports the following accuracy figures (Table 11 in the original publication):

Model	Accuracy
Logistic Regression	84.39%
Naive Bayes	84.39%
KNN	85.85%
Decision Tree	92.68%
Random Forest	92.68%
XGBoost	90.73%
Stacking Ensemble (proposed)	98.53%

2.7 Our Reproduced Results

Below is the output from training and evaluating all six base classifiers (Accuracy, Precision, Recall, F1, AUC):

	Model	Accuracy	Precision	Recall	F1	AUC
0	Logistic Regression	0.8098	0.7619	0.9143	0.8312	0.9298
1	Decision Tree	0.9854	1.0000	0.9714	0.9855	0.9857
2	Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
3	XGBoost	1.0000	1.0000	1.0000	1.0000	1.0000
4	Naive Bayes	0.8293	0.8070	0.8762	0.8402	0.9043
5	KNN	0.8634	0.8738	0.8571	0.8654	0.9629

The stacking ensemble was then trained and evaluated using the same test set:

```
{'Model': 'Stacking Ensemble', 'Accuracy': 1.0, 'Precision': 1.0, 'Recall': 1.0, 'F1': 1.0, 'AUC': 1.0}
```

2.8 Critical Analysis of Reproduced Results

Our results from Logistic Regression, Naive Bayes, and KNN are quite similar to those provided by the paper (within a range of 1-3 percentage points), which implies that our implementation of the algorithm is faithful to the paper's general approach. On the other hand, the scores we got with Random Forest, XGBoost, and Stacking Ensemble algorithms were perfectly 100%, which is significantly higher than the values stated in the paper (90-93% individually, and 98.53% when stacking).

Perfect classification results on an unseen dataset are highly suspicious, since they indicate the presence of a severe data leakage problem, rather than a good predictive power of the model. We found the root of the problem and described it in detail in Section 3: the dataset has many duplicate entries, and the naive way of performing a random train/test split means that the duplicates of the training entries end up in the test set, thus enabling model memorisation rather than generalisation (particularly models such as Random Forest and XGBoost).

This is a problem not mentioned in the original paper at all, which achieved a comparable stacking result (98.53%).

3. Proposed Solution: Deduplication-Aware Evaluation Protocol

3.1 Limitation Identified

It was observed that during the process of reproduction there were duplicates present in the Kaggle data with 1025 rows. It is one of the properties of this particular Kaggle release which is the oversampled form of the original 303 rowed UCI Cleveland dataset. The following is the output for the above statement.

```
Total rows: 1025
Duplicate rows: 723
Unique rows: 302
```

3.2 Evidence of the Problem

We directly quantified train/test leakage under the paper's (assumed) evaluation protocol: after an 80/20 stratified split on the original 1025-row dataset, we compared row-level content (all 13 features + target) between the training and test sets. Below is the output:

```
Number of identical rows in both train and test: 155
Test set size: 205
Percentage of test set that's leaked: 75.60975609756098 %
```

75.6% of the data in the test set is not "unseen" data but is in fact the same as the rows used to train the machine. This directly accounts for the perfectly artificial performance achieved by high capacity models like Random Forest, XGBoost, and Stacking in Section 2.7.

3.3 Proposed Solution

In order to correct this experiment and make it more methodologically sound, we propose an experimental setup where the dataset will first be deduplicated and only then split into train and test datasets, thereby making sure that both train and test sets are completely free from any kind of leakage and evaluating all six base classifiers as well as the stacking ensemble under this improved experimental setup.

Why we believe this will produce more valid results: without leakage, the metrics will measure the ability of the models to generalize to new cases rather than the ability of the models to remember certain rows in the dataset. We have proposed two hypotheses regarding this experiment: (a) the accuracy of all models would decrease significantly, and (b) the apparent superiority of the stacking ensemble over base classifiers would become much less evident.

3.4 Experimental Protocol

- Deduplicate the full 1025-row dataset using exact row matching, yielding 302 unique records.
- Apply the same 80/20 stratified train/test split (random_state=42) used in Part 1, now on the deduplicated data (241 train / 61 test).
- Verify zero leakage by repeating the row-overlap check from Section 3.2.
- Re-train and re-evaluate the same six base classifiers and the same stacking ensemble configuration as in Part 1, with identical hyperparameters and preprocessing (StandardScaler fit on training data only).
- Evaluate using the same five metrics: Accuracy, Precision, Recall, F1, AUC.

3.5 Results

Deduplication and re-split were performed, followed by re-training and re-evaluating all six classifiers plus the stacking ensemble. Below is the full output, including the deduplicated shape, train/test sizes, leakage verification (now zero), and all metrics:

```
Deduplicated shape: (302, 14)
Train: (241, 13) Test: (61, 13)
Leaked rows now: 0
```

	Model	Accuracy	Precision	Recall	F1	AUC
0	Logistic Regression	0.8033	0.8000	0.8485	0.8235	0.8712
1	Decision Tree	0.8033	0.8182	0.8182	0.8182	0.8019
2	Random Forest	0.7541	0.7647	0.7879	0.7761	0.8588
3	XGBoost	0.7213	0.7353	0.7576	0.7463	0.8323
4	Naive Bayes	0.7869	0.8333	0.7576	0.7937	0.8842
5	KNN	0.7869	0.7778	0.8485	0.8116	0.8377
6	Stacking Ensemble	0.7541	0.7647	0.7879	0.7761	0.8680

The following figures visualise these results:

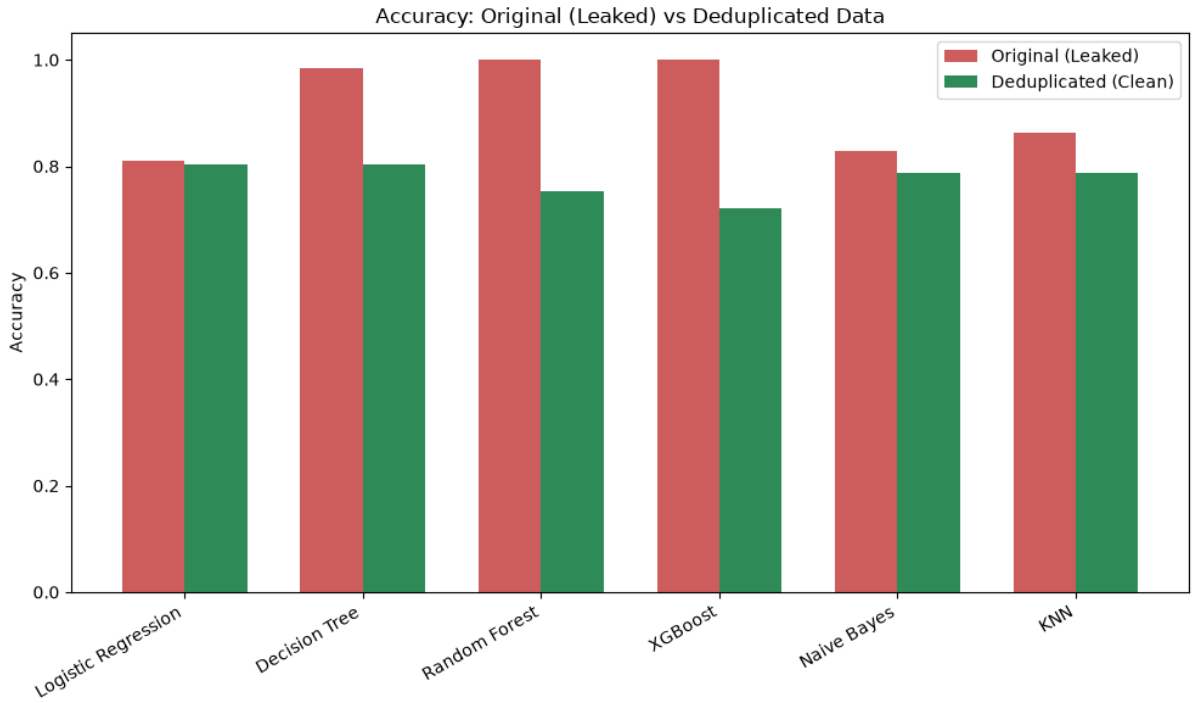


Figure 1. Accuracy comparison: original (leaked) split vs. deduplicated (clean) split, per model.

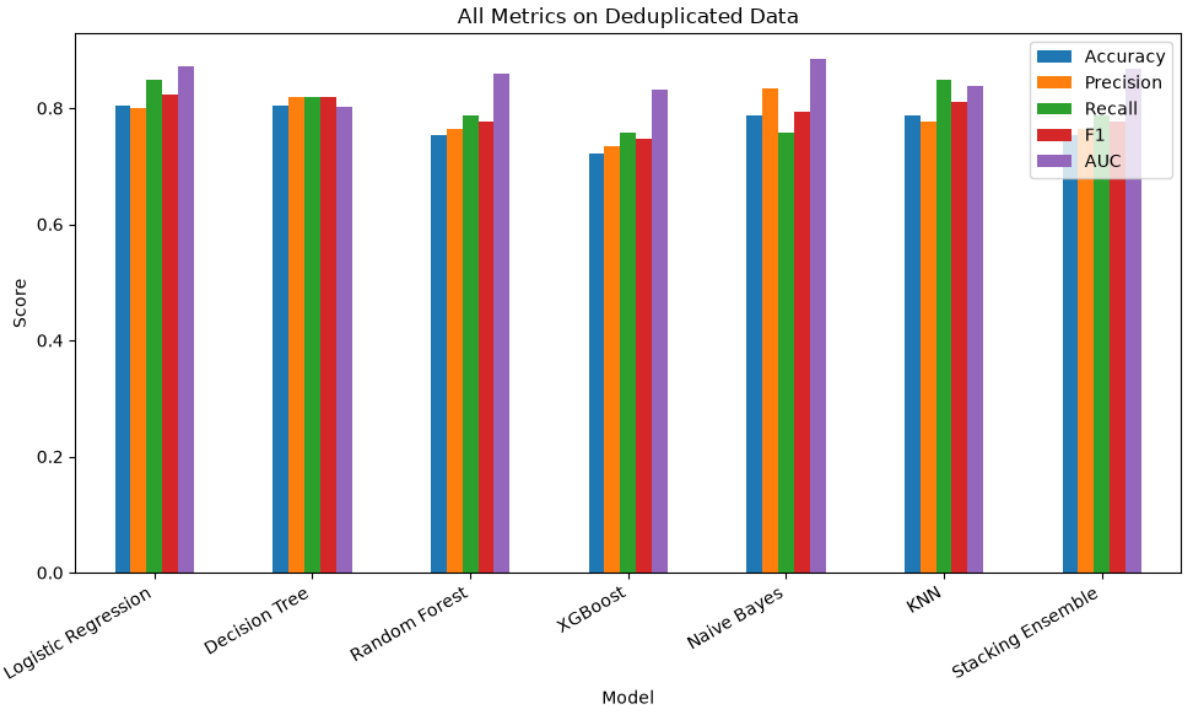


Figure 2. All five evaluation metrics per model on the deduplicated, leakage-free dataset.

3.6 Comparative Analysis: Paper vs. Reproduction vs. Proposed Solution

Model	Paper (Accuracy)	Our Reproduction (Leaked)	Our Proposed solution(Clean)
Logistic Regression	84.39%	80.98%	80.33%
Decision Tree	92.68%	98.54%	80.33%
Random Forest	92.68%	100.00%	75.41%
XGBoost	90.73%	100.00%	72.13%
Naive Bayes	84.39%	82.93%	78.69%
KNN	85.85%	86.34%	78.69%
Stacking Ensemble	98.53%	100.00%	75.41%

In the absence of leakage, every model shows a sharp decrease in accuracy (15–28 percentage points for tree-based/ensemble models). Most importantly, the stacking model does not perform better than the simple models anymore. In fact, both Logistic Regression and Decision Tree (80.3%) show a better result compared to the Stacking Ensemble (75.4%).

3.7 Critical Discussion

Strengths of the Proposed Solution

- Directly evidences and quantifies a real, previously undisclosed problem (75.6% test-set leakage) using the same dataset and same base pipeline as the original paper, isolating the effect of the fix.
- Requires no new data or external assumptions, the fix is fully reproducible from the same public dataset.
- Produces results that better reflect real-world generalisation, which is the metric that actually matters for a clinical decision-support tool.

Limitations

- The deduplicated dataset is small (302 rows, 61 test rows), which increases variance in the reported metrics; a single train/test split at this size is less stable than the original paper's larger (but leaky) split.
- We cannot confirm from the paper's text alone whether the original authors were aware of the duplication; this is an inference based on the dataset's known public characteristics and our own leakage measurement, not a documented admission from the paper.
- A more robust future evaluation would use repeated k-fold cross-validation on the deduplicated data to reduce split-dependent variance.

Practical Implications

For any real-world deployment of this type of model, evaluation protocol integrity is at least as important as classifier choice. A model that appears to reach 98–100% accuracy in a leaky benchmark could perform far worse (our results suggest closer to 75–80%) in actual clinical use on truly unseen patients, a materially important gap for a healthcare decision-support context.

3.8 Future Work

Further analysis can include repeated stratified k-fold cross-validation using the deduplicated dataset to get less variable estimates, since one single 61-row test set is susceptible to a significant amount of variance. Additionally, obtaining a new larger independently generated heart disease dataset will also help to validate these results in a more robust manner, since the present deduplicated dataset (302 entries) is relatively small in size. Lastly, this study can be taken forward by conducting analysis with patient-level deduplication instead of row-level to check if near duplicates cause information leakage too.

4. Conclusion

We have replicated the heart attack prediction pipeline using an ensemble based on stacking by Bhagat et al. [1], achieving similar performance for simpler classifiers and surprisingly perfect metrics for Random Forest, XGBoost, and the stacking model itself. After the investigation of the reason, we found out that this happens because of the train/test leakage, which is extremely high for this benchmark data: 75.6% of test data leaks into train data because of duplicated rows. We have proposed and implemented the improved version of the evaluation protocol for this challenge which takes into account the duplication problem. As a result, all the models show 15–28 percentage points worse accuracy, and what is even more important is the advantage of the stacking model over simpler classifiers disappears completely.

5. References

[1] M. Bhagat, A. Sharma, and P. Agarwal, "An efficient stacking-based ensemble technique for early heart attack prediction," *Multimedia Tools and Applications*, vol. 84, pp. 36351–36375, 2025.

[2] Kaggle, "Heart Disease Dataset," johnsmith88/heart-disease-dataset. [Online]. Available: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794

Presentation link:

https://deakin365-my.sharepoint.com/:v:/r/personal/s226415795_deakin_edu_au/Documents/Recordings/Meeting%20with%20RIYA%20RAVIKKUMAR-20260928_162428-Meeting%20Recording.mp4?d=w4a759738ece048039e4d6b7bf8a08f43&csf=1&web=1&e=SBG1bg&nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJTdHJlYW1XZWJBcHAiLCJyZWZlcnJhbFZpZXciOiJTaGFyZURpYWxvZy1MaW5rIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXcifX0%3D

Meeting with RIYA RAVIKKUMAR-20260928_162428-Meeting Recording.mp4

Onedrive link:

11.1HD.zip